

INFORME DE ARQUITECTURA DE SOFTWARE:

Backend

1. INTRODUCCIÓN

El siguiente informe describe la arquitectura de software elegida para el desarrollo del backend del sistema **invernadero inteligente**. Esta etapa (Backend) tiene por objetivo el almacenamiento y procesamiento de los datos provenientes de la capa de firmware. A su vez esta etapa es crucial, ya que incluye toda la lógica de negocio la cual es necesaria mantener desacoplada de toda la infraestructura.

2. INFRAESTRUCTURA (STACK TECNOLÓGICO)

Antes de definir la arquitectura detallada del sistema, es fundamental establecer el conjunto de tecnologías que darán soporte a la aplicación. Las herramientas seleccionadas garantizan un entorno robusto, tipado y preparado para el manejo de datos en tiempo real provenientes del invernadero.

NestJS (Framework de Backend):

Se seleccionó NestJS como framework principal debido a su arquitectura nativa basada en TypeScript, lo que aporta tipado estático al proyecto. Su sistema de módulos e inyección de dependencias encaja con los principios de la Arquitectura Hexagonal, facilitando el desacoplamiento de componentes.

PostgreSQL (Capa de persistencia)

Para el almacenamiento de datos se optó por PostgreSQL, un sistema de gestión de bases de datos relacionales.

TypeORM (Type Relational Mapping)

Como puente entre la aplicación y la base de datos se utilizará TypeORM. Este ORM permite interactuar con PostgreSQL utilizando clases y decoradores de TypeScript, lo que reduce la escritura de SQL manual y acelera el desarrollo.

Swagger y OpenAPI (Documentación)

La comunicación entre el backend, la capa de firmware y el frontend requiere contratos de interfaz claros. Swagger, bajo la especificación OpenAPI, se integrará de forma automática en NestJS para generar una documentación interactiva y auto-actualizable de los endpoints de la API.

3. ARQUITECTURA HEXAGONAL

La capa de backend se estructurará bajo los lineamientos de la arquitectura hexagonal. La decisión técnica se fundamenta en la necesidad crítica de aislar las reglas de negocio de los detalles de infraestructura (frameworks, controladores y bases de datos).

Fundamentos de la arquitectura hexagonal y su impacto dentro del sistema.

Independencia Tecnológica y Desacoplamiento

El dominio de nuestro sistema no poseerá dependencias directas con el framework (NestJS) ni la comunicación directa con la base de datos.

Esto garantiza que si en el futuro se requiere migrar de tecnologías, como el protocolo de red de HTTP a MQTT, o el motor de bases de datos de PostgreSQL a MongoDB, las reglas de negocio no se verán afectadas

Inversión de Dependencias (DIP):

Esto significa que las capas externas de infraestructura dependen del dominio. Dentro del proyecto el dominio está definido por los servicios (archivos del tipo *.services.ts), los adaptadores tanto de entrada como de salida dependen exclusivamente de estos servicios y no al revés.

Mecanismo de Puertos y Adaptadores:

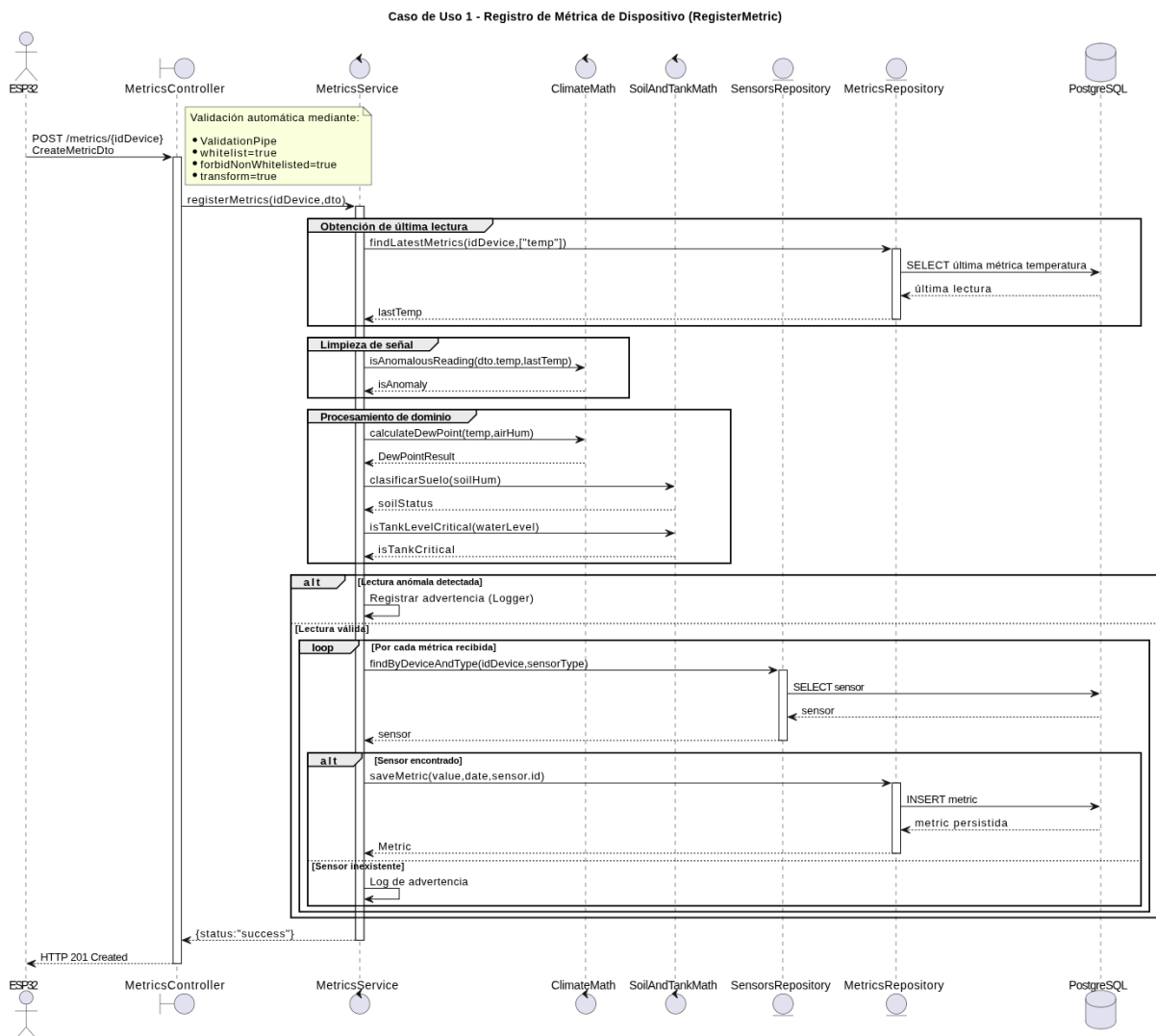
- **Adaptadores de Entrada:** Encabezados por las clases controladoras (Controllers), estarán encargados de recibir solicitudes externas (HTTP request) y validar la estructura de los datos provenientes de la capa de firmware.
- **Puertos de entrada:** Estarán definidos mediante interfaces de typescript donde se declararan los métodos para la comunicación entre los controladores con los servicios.
- **Puertos de salida:** Estarán definidos mediante interfaces de typescript, pero servirán definir los métodos que el dominio utilizara para la comunicación con la capa de persistencia.
- **Adaptadores de Salida:** Encabezados por la capa de persistencia sobre contenedores Docker con PostgreSQL. Implementarán las operaciones de lectura y escritura exigidas por el dominio abstrayendo la complejidad de las consultas SQL mediante TypeORM.

4. CASOS DE USO

El comportamiento y los flujos de información del sistema han sido particionados en Casos de Uso específicos y autónomos que residen en la capa de dominio del backend.

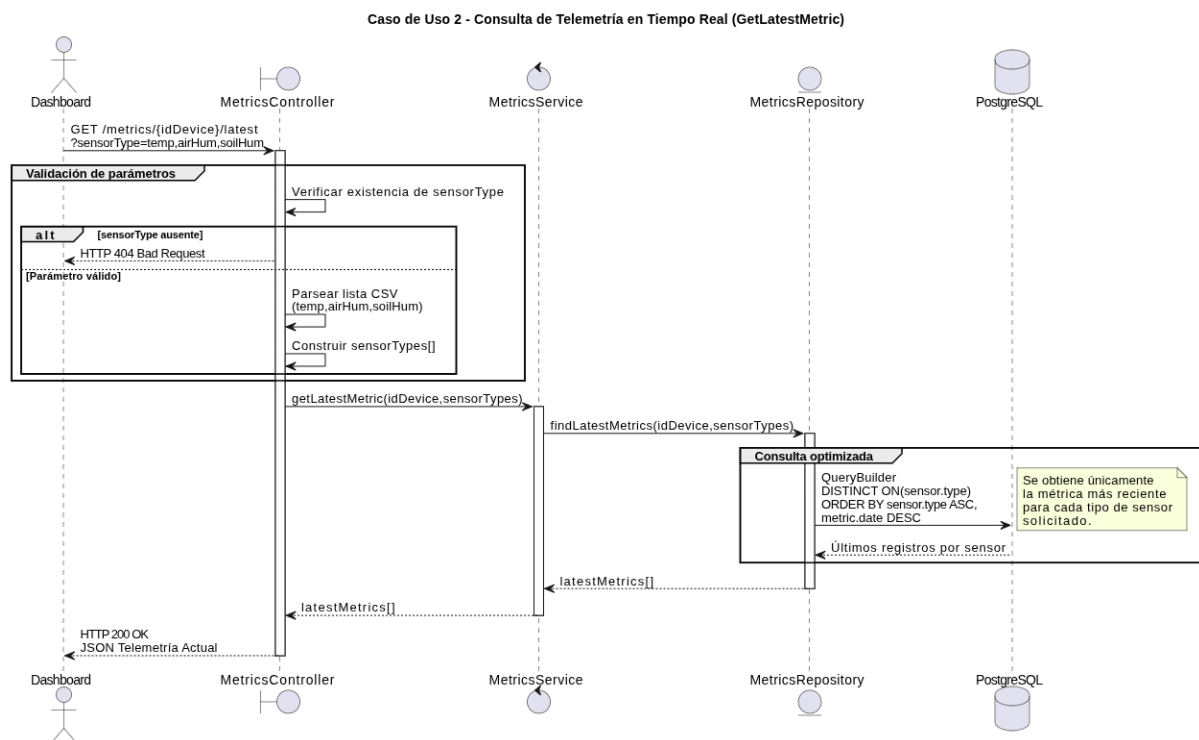
Caso de Uso 1: Registro de Métrica de Dispositivo (*RegisterMetric*)

- **Actor Principal:** Microcontrolador ESP32 (Hardware Remoto).
- **Propósito:** Validar, procesar e incorporar una nueva lectura de telemetría física al histórico del sistema.
- **Flujo de Ejecución:**



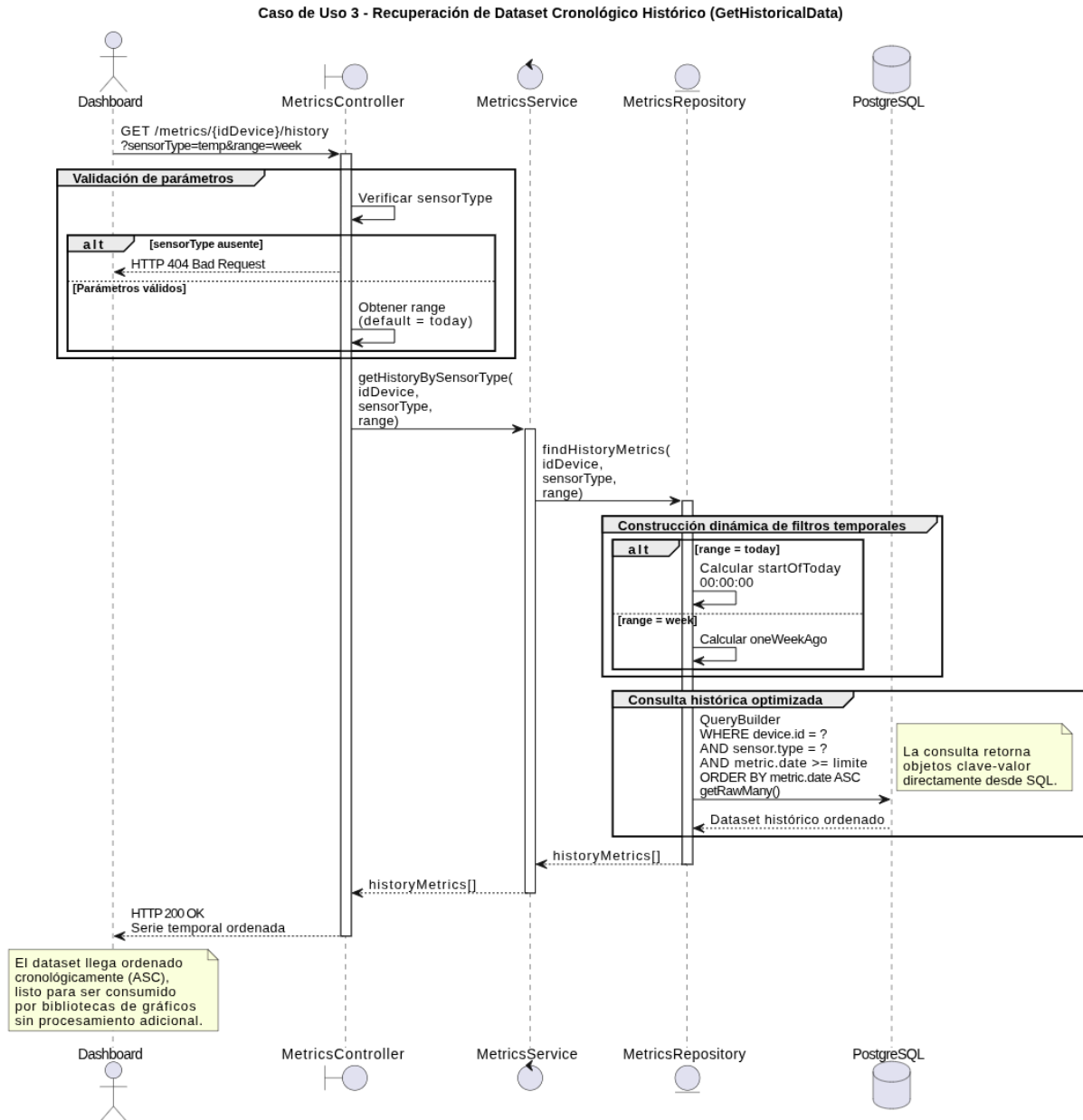
Caso de Uso 2: Consulta de Telemetría en Tiempo Real (GetLatestMetric)

- **Actor Principal:** Cliente Frontend (Aplicación Dashboard Next.js).
- **Propósito:** Proveer el último estado conocido absoluto del invernadero para alimentar las visualizaciones rápidas y los widgets dinámicos.
- **Flujo de Ejecución:**



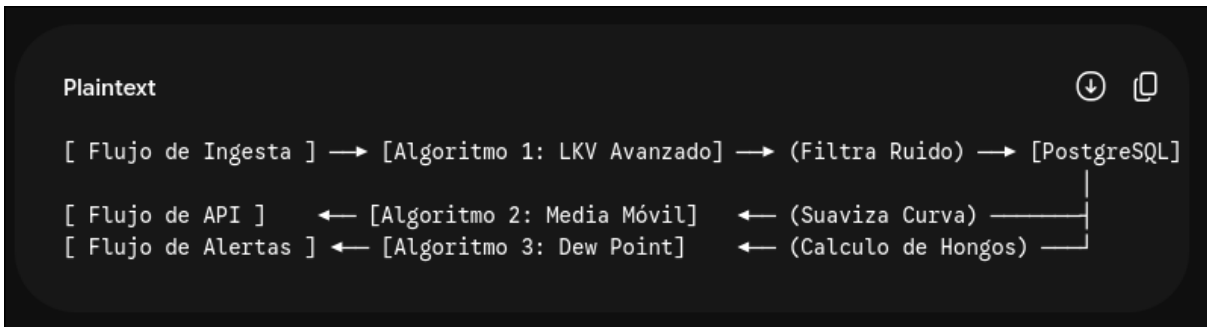
Caso de Uso 3: Recuperación de Dataset Cronológico Histórico (GetHistoricalData)

- **Actor Principal:** Cliente Frontend (Módulos Analíticos y Gráficos).
- **Propósito:** Suministrar series de tiempo ordenadas cronológicamente para evaluar tendencias agrícolas agregadas.
- **Flujo de Ejecución:**



5. ALGORITMOS DE PROCESAMIENTO

Se implementarán diferentes algoritmos para el procesamiento de datos climáticos, los cuales estarán en una carpeta separada del resto de carpetas de la infraestructura. Estos algoritmos se encargaran de realizar transformaciones de los datos obtenidos en valor para nuestro sistema. Nos hemos encargado de seleccionar los siguientes en base a los requerimientos funcionales propuestos con anterioridad al desarrollo.



Algoritmo 1: Filtro Estadístico de Anomalías y Ruido Eléctrico (LKV Avanzado)

Tiene como objetivo mitigar las anomalías de lectura e ingresos de falsos positivos provocados por ruidos eléctricos o caídas de tensión nominal en los sensores analógicos/digitales del microcontrolador ESP32.

$$\text{Variación} = \frac{|V_{\text{actual}} - V_{\text{anterior}}|}{V_{\text{anterior}}}$$

Algoritmo 2: Agregación por Ventana Temporal y Suavizado (Media Móvil Simple)

Tiene como propósito compactar los datos enviados hacia la interfaz de usuario en [Next.js](#). Debido a que los datos almacenados tienen margen mínimo de diferencia temporal (5 segundos), se haría imposible visualizar los datos correctamente en los gráficos correspondientes a cada sensor.

Algoritmo 3: Cálculo del Punto de Rocío (Dew Point) y Clasificación de Riesgo Biótico

Tiene como propósito evaluar las condiciones de saturación de humedad en el aire para predecir la condensación física de agua sobre los cultivos, previniendo la proliferación de patógenos (hongos).

Este algoritmo evalúa la diferencia termodinámica (Tambiente - Trocio). Si el margen es igual o menor a 2.0 C, el algoritmo transforma la salida en una alerta categorizada como **RIESGO_DE_HONGOS**, permitiendo al Dashboard renderizar advertencias críticas en tiempo real.

Algoritmo 4: Clasificación del Estado de Humedad del Suelo y Regulación Hídrica.

Tiene como propósito evaluar de manera continua el nivel de humedad en la tierra. Este algoritmo procesa el valor porcentual recibido por el sensor de suelo y lo segmenta en tres

estados operativos: si el valor es menor a 30%, devuelve el estado “Seco- Requiere riego”; si se encuentra en un rango inclusivo entre el 30% y el 70%, dictamina un estado “Óptimo”; y para cualquier valor superior, categoriza el suelo como “Inundado”. Esto permite al dashboard renderizar el estado del sustrato en tiempo real y condicionar de forma segura el encendido del riego.

Algoritmo 5: Monitoreo del Nivel Crítico del Tanque y Protección de Actuadores.

Tiene como objetivo evaluar la disponibilidad del recurso hídrico en la reserva principal, previniendo fallas mecánicas catastróficas y el quemado de la bomba de agua por funcionamiento en vacío.

Este algoritmo evalúa de forma booleana el volumen porcentual del tanque de almacenamiento. Si el nivel es estrictamente menor al 15%, el algoritmo retorna un valor verdadero, **true**, de lo contrario, retorna un valor falso, **false**.